

Assignment_5

Keerthi Priya Jakku

2026-04-23

Necessary packages

```
library(stats)
library(factoextra)
```

```
## Loading required package: ggplot2
```

```
## Welcome to factoextra!
```

```
## Want to learn more? See two factoextra-related books at https://www.datanovia.com/en/product/practical-guide-to-principal-component-methods-in-r/
```

```
library(cluster)
library(readr)
library(ggplot2)
```

Load Data

```
cereals<-read.csv("~/Desktop/R documents/Assignment files/FML_Assignment 5/Cereals.csv")
```

```
head(cereals)
```

```
##           name mfr type calories protein fat sodium fiber carbo
## 1      100%_Bran  N   C        70         4  1   130   10.0   5.0
## 2 100%_Natural_Bran Q   C       120         3  5    15    2.0   8.0
## 3      All-Bran   K   C        70         4  1   260    9.0   7.0
## 4 All-Bran_with_Extra_Fiber K   C        50         4  0   140   14.0   8.0
## 5      Almond_Delight R   C       110         2  2   200    1.0  14.0
## 6 Apple_Cinnamon_Cheerios G   C       110         2  2   180    1.5  10.5
##  sugars potass vitamins shelf weight cups  rating
## 1      6      280       25     3      1 0.33 68.40297
## 2      8      135        0     3      1 1.00 33.98368
## 3      5      320       25     3      1 0.33 59.42551
## 4      0      330       25     3      1 0.50 93.70491
## 5      8       NA       25     3      1 0.75 34.38484
## 6     10       70       25     1      1 0.75 29.50954
```

Removing missing values

```
cereals <- na.omit(cereals)
```

Normalizing data

```
numeric <-sapply(cereals, is.numeric)
cereals_numeric <- cereals[, numeric]
cereals_normalized <- scale(cereals_numeric)
cereals_normalized <- as.data.frame(cereals_normalized)
```

```

categorical_col <- cereals[, 1:3]
cereals_numericalscol <- cereals_normalized[, 1:13]
cereals_final <- cbind(categorical_col, cereals_numericalscol)
cereals_final <- as.data.frame(cereals_final)
cereals_final <- na.omit(cereals_final)

infinite_values <- sapply(cereals_final, function(x) any(is.infinite(x)))
total_infinite_values <- sum(infinite_values)
if (total_infinite_values > 0) {
  cat("There are", total_infinite_values, "columns with infinite values in the dataset.\n")
} else {
  cat("No infinite values found in the dataset.\n")
}

## No infinite values found in the dataset.

nan_columns <- sapply(cereals_final, function(x) any(is.nan(x)))
total_nan_columns <- sum(nan_columns)
if (total_nan_columns > 0) {
  cat("There are", total_nan_columns, "columns with NaN values in the dataset.\n")
} else {
  cat("No NaN values found in the dataset.\n")
}

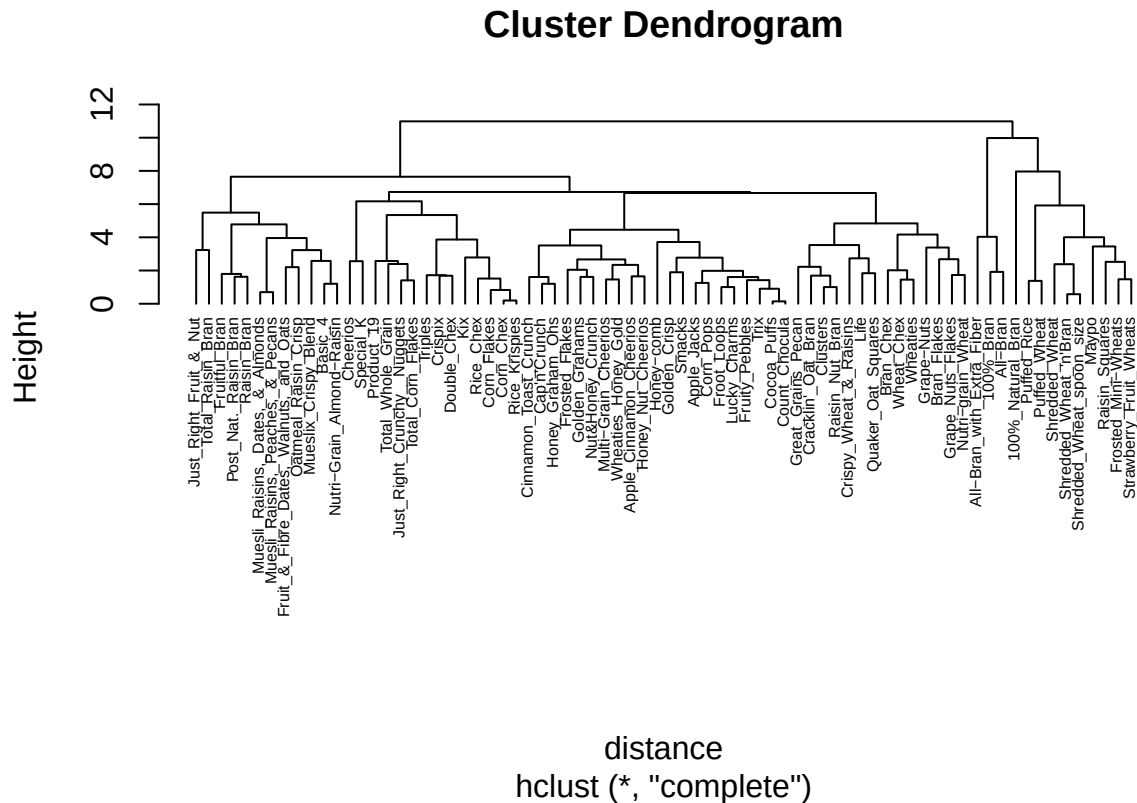
## No NaN values found in the dataset.

Hierarchial Clustering using Euclidean distance

rownames(cereals_normalized) <- categorical_col[, 1]
distance <- dist(cereals_normalized, method= "euclidean")
Hierarchialclustering <- hclust(distance, method = "complete")

plot (Hierarchialclustering, cex = 0.5, hang= -1)

```



Using Agnes

```
agnes_single <- agnes(cereals_normalized, metric = "euclidean", method = "single")
agnes_complete <- agnes(cereals_normalized, metric = "euclidean", method = "complete")
agnes_average <- agnes(cereals_normalized, metric = "euclidean", method = "average")
agnes_ward <- agnes(cereals_normalized, metric = "euclidean", method = "ward")
```

linkages with Agglomerative Coefficient

```
print(agnes_single$ac)
```

```
## [1] 0.6067859
```

```
print(agnes_complete$ac)
```

```
## [1] 0.8353712
```

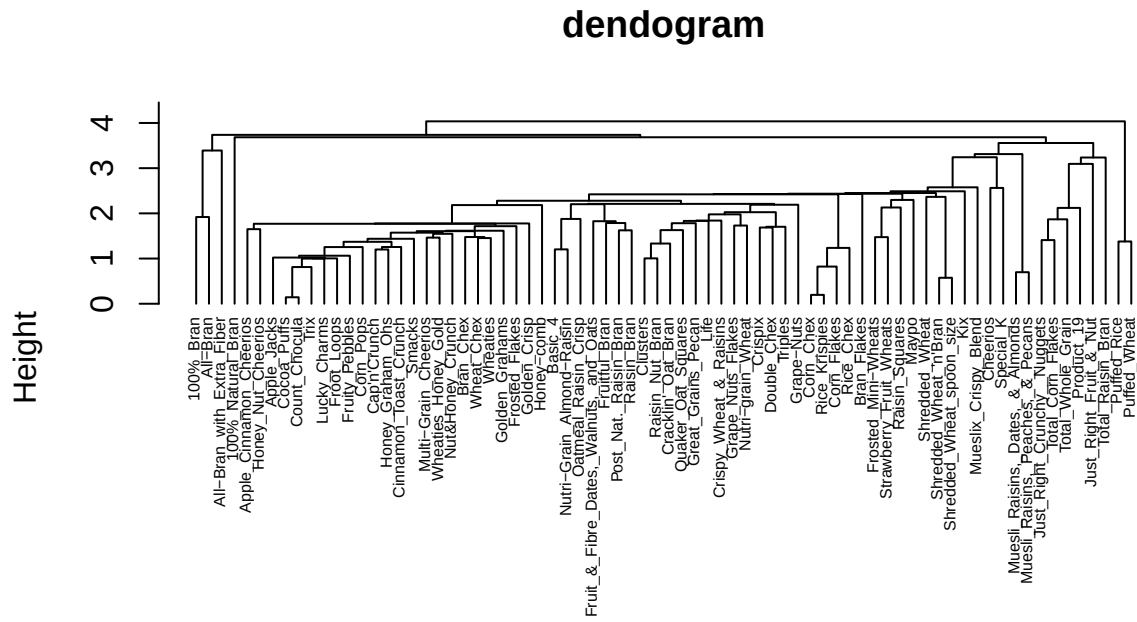
```
print(agnes_average$ac)
```

```
## [1] 0.7766075
```

```
print(agnes_ward$ac)
```

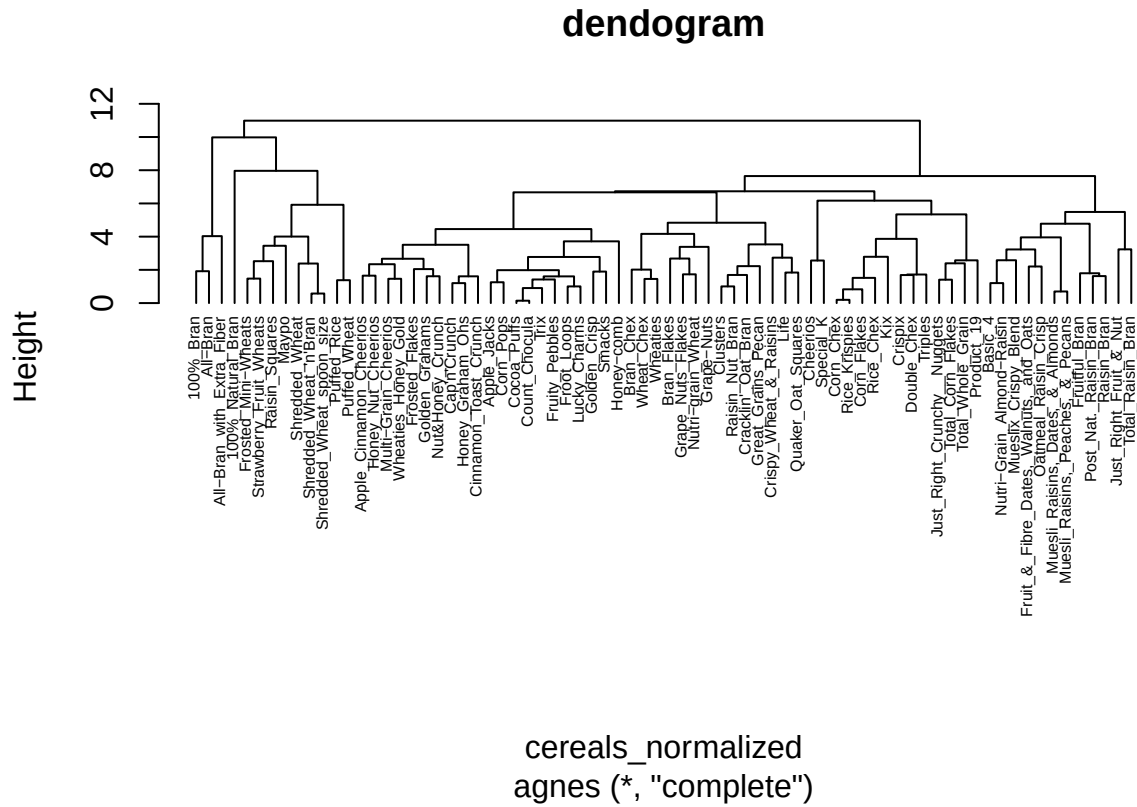
```
## [1] 0.9046042
```

```
pltree(agnes_single, cex=0.5, hang=-1, main= "dendogram")
```



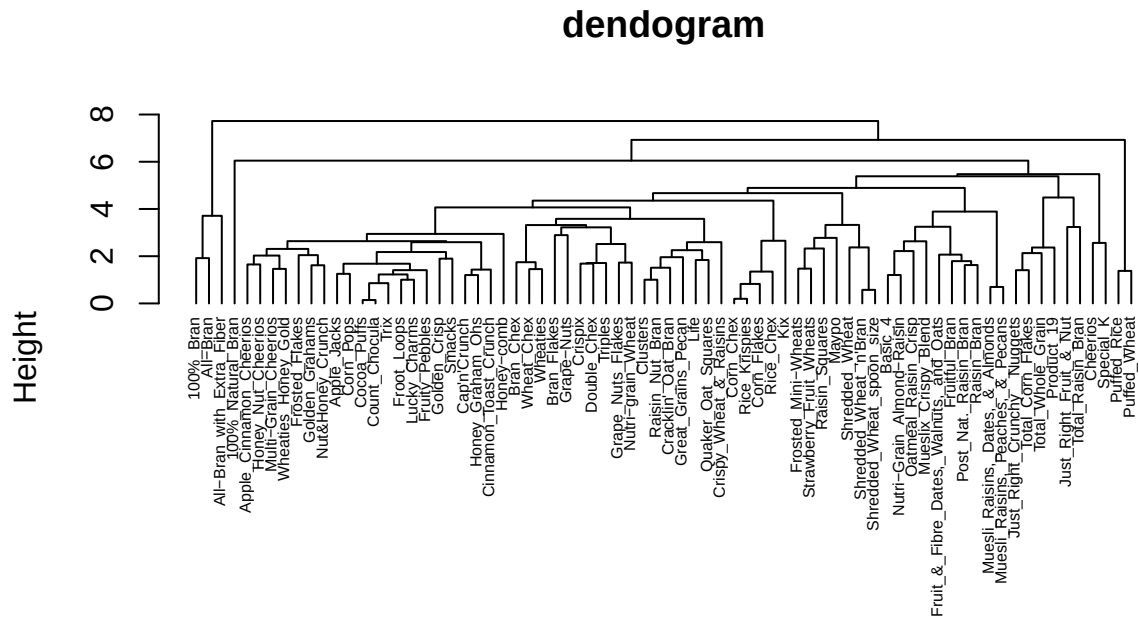
cereals_normalized
agnes (*, "single")

```
pltree(agnes_complete, cex=0.5, hang=-1, main= "dendrogram")
```



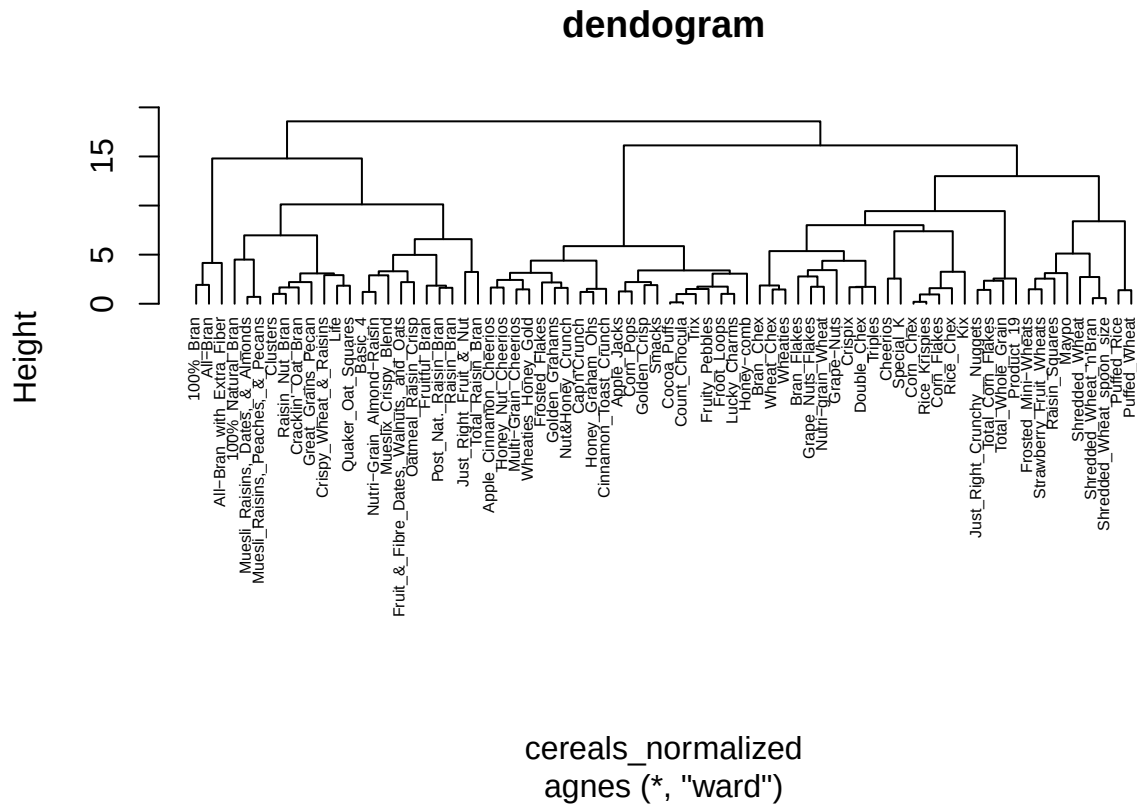
cereals_normalized
agnes (*, "complete")

```
pltree(agnes_average, cex=0.5, hang=-1, main= "dendrogram")
```

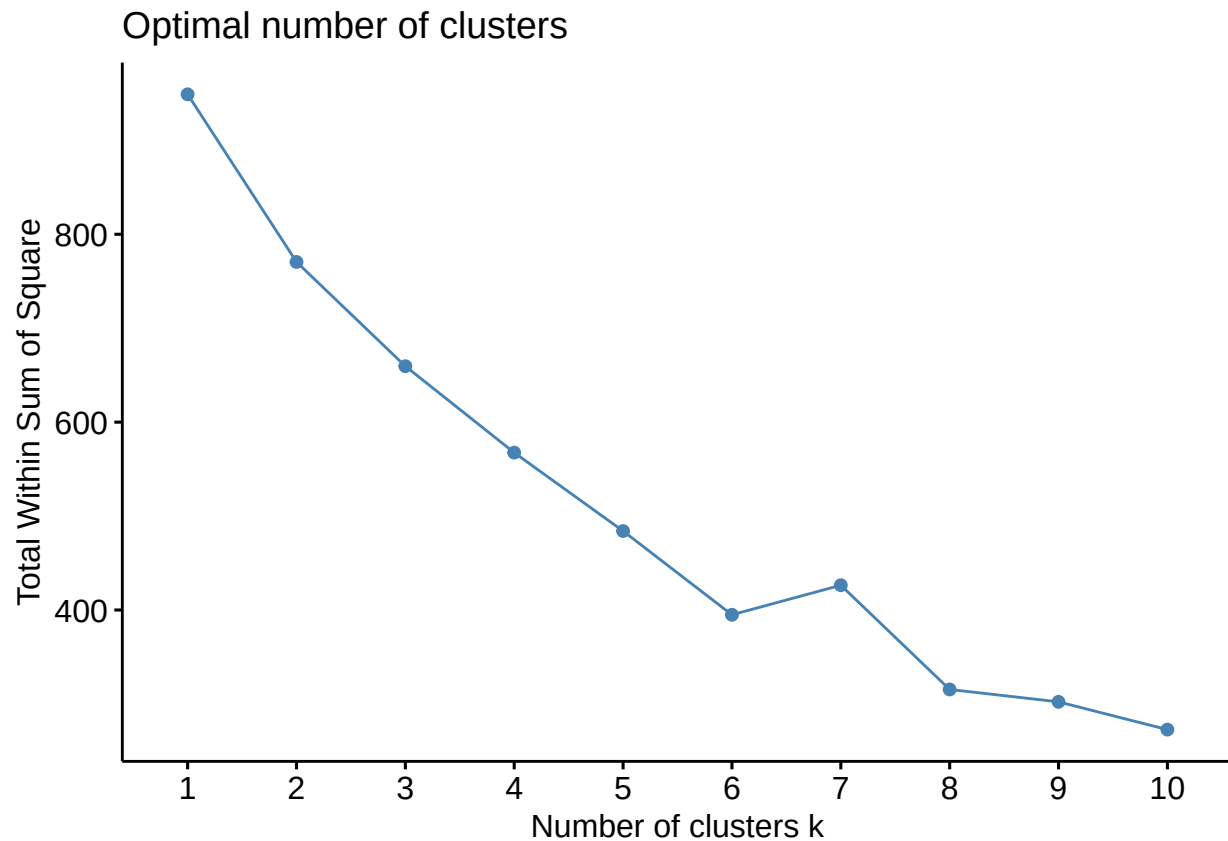


cereals_normalized
agnes (*, "average")

```
pltree(agnes_ward, cex=0.5, hang=-1, main= "dendrogram")
```



```
set.seed(123)
fviz_nbclust(as.data.frame(cereals_normalized), FUN = kmeans, method = "wss")
```



optimal number of clusters k=5

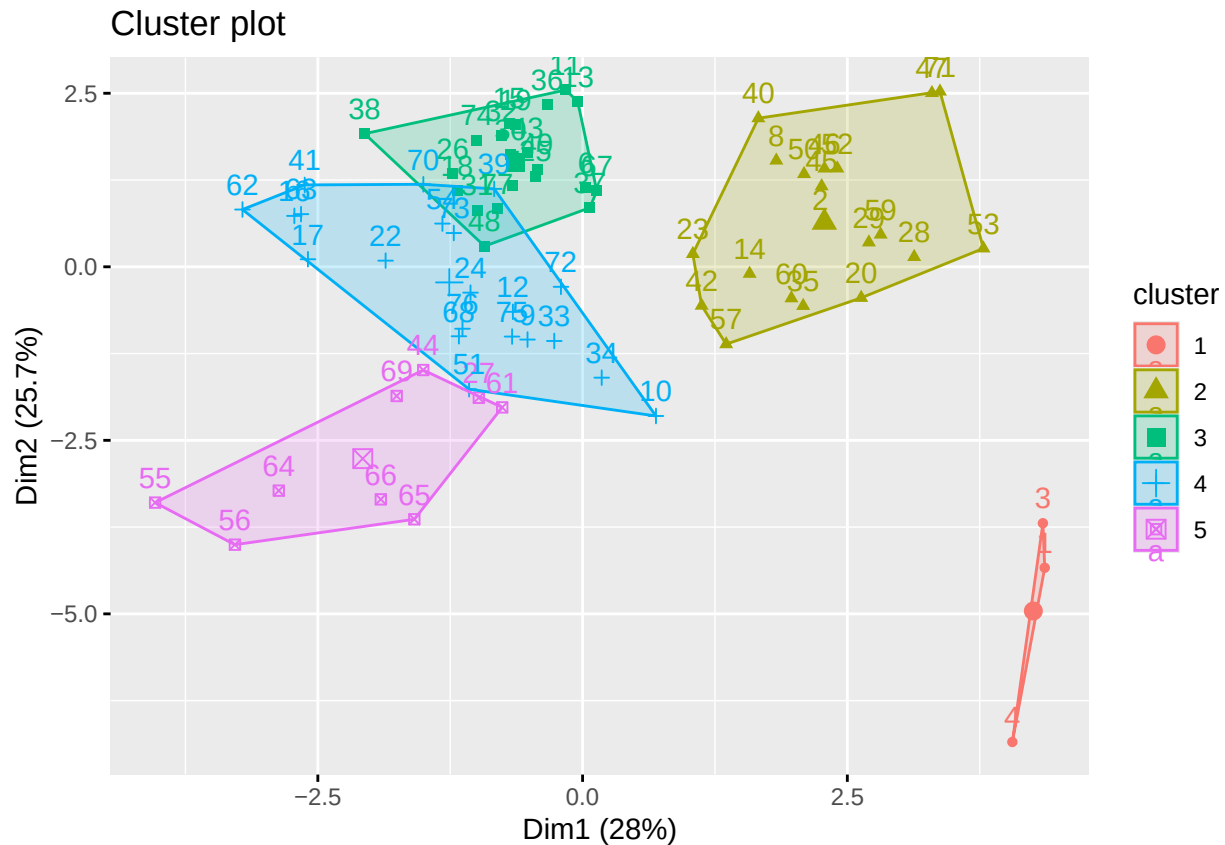
```
number_of_clusters <- cutree(agnes_ward, k = 5)
number_clusters <- length(unique(number_of_clusters))
print(number_clusters)
```

```
## [1] 5
```

Structure of clusters and their stability

Visualization of clusters

```
combined_data <- as.data.frame(cbind(cereals_final, number_of_clusters))
numeric_data <- combined_data[, sapply(combined_data, is.numeric)]
fviz_cluster(list(data = numeric_data, cluster = number_of_clusters))
```



Centroids

```
centroids <- t(sapply(unique(number_of_clusters), function(cluster) {
  colMeans(numeric_data[number_of_clusters == cluster, ])
}))

print(centroids)
```

##	calories	protein	fat	sodium	fiber	carbo
## [1,]	-2.2018711	1.38174776	-3.310734e-01	0.17279012	3.64131237	-2.0718749
## [2,]	0.8553248	0.59163927	9.435592e-01	-0.08898011	0.38141771	-0.2003584
## [3,]	0.1978117	-0.91996886	1.057355e-17	0.12101140	-0.66198437	-0.5423583
## [4,]	-0.1621407	0.18662567	-4.729620e-01	0.77112209	-0.21003997	0.9626860
## [5,]	-1.2499969	-0.06420242	-8.828625e-01	-1.94150793	-0.02664224	0.1551013

##	sugars	potass	vitamins	shelf	weight	cups
## [1,]	-0.7894824	2.9837813	-0.18184220	0.9419715	-0.2008324	-1.8452553
## [2,]	0.5143002	0.7475659	0.09849786	0.8217889	0.9235649	-0.5477863
## [3,]	0.9583619	-0.7415648	-0.18184220	-0.6604628	-0.2008324	0.2779676
## [4,]	-0.8659505	-0.3485391	0.45893508	-0.1453946	-0.2008324	0.4577648
## [5,]	-1.0953551	-0.1122758	-0.80482011	-0.2598542	-1.0482044	0.1156788

##	rating	number_of_clusters
## [1,]	2.2426479	1
## [2,]	-0.2928786	2
## [3,]	-0.9636465	3
## [4,]	0.2916795	4
## [5,]	1.4712151	5

Partitions


```
set.seed(123)
indices <- sample(nrow(cereals_normalized), size = 0.5 * nrow(cereals_normalized))
Partition_A <- cereals_normalized[indices, ]
Partition_B <- cereals_normalized[-indices, ]
```

```
agnes_single_A <- agnes(Partition_A, method = "single")
agnes_average_A <- agnes(Partition_A, method = "average")
agnes_complete_A <- agnes(Partition_A, method = "complete")
agnes_ward_A <- agnes(Partition_A, method = "ward")
```

Partition A - Agglomerative Coefficient

```
agnes_single_A$ac
```

```
## [1] 0.5677606
```

```
agnes_average_A$ac
```

```
## [1] 0.68553
```

```
agnes_complete_A$ac
```

```
## [1] 0.7696915
```

```
agnes_ward_A$ac
```

```
## [1] 0.8467852
```

```
historicalclustering_A <- agnes(Partition_A, method = "ward")
cut_A <- cutree(historicalclustering_A, k = 5)
centroids_A <- aggregate(Partition_A, by = list(cut_A), FUN = mean)

assign_Centroid <- function(observation, centroids) {
  distances <- apply(centroids[, -1], 1, function(centroid) sum((observation - centroid)^2))
  return(which.min(distances))
}
```

```
cluster_assignments_B <- apply(Partition_B, 1, assign_Centroid, centroids_A)
```

```
historicalclustering_full <- agnes(cereals_normalized, method = "ward")
cut_full <- cutree(historicalclustering_full, k = 5)
```

```
table(cluster_assignments_B, cut_full[-indices])
```

```
##
## cluster_assignments_B  1  2  3  4  5
##                1  2  1  0  4  5
##                2  1  2  0  0  0
##                3  0  1 10  0  0
##                4  0  0  0  7  0
##                5  0  4  0  0  0
```

Optimal cluster for a healthy cereal

```
cereals_healthy <- cereals[complete.cases(cereals), ]
agnes_ward_healthy <- agnes(cereals_healthy[, -c(1:3)], method = "ward")
historicalclustering_ward_healthy <- as.hclust(agnes_ward_healthy)
clustering <- cutree(historicalclustering_ward_healthy, k = 5)
```

```

cluster_means <- aggregate(cereals_healthy[, -c(1:3)],
                           by = list(cluster = clustering), FUN = mean)

healthy_cluster_index <- which.max(cluster_means$fiber + cluster_means$protein
                                   - cluster_means$sugars - cluster_means$fat)
healthy_cluster <- cluster_means[healthy_cluster_index, ]

print(cluster_means)

##   cluster calories  protein      fat  sodium    fiber   carbo    sugars
## 1         1 112.1429 3.357143 1.428571 172.5000 5.6071429 12.03571 9.142857
## 2         2  86.0000 2.500000 0.600000   3.0000 2.1000000 14.60000 2.900000
## 3         3 110.3226 2.290323 1.161290 187.5806 1.5645161 14.91935 7.645161
## 4         4 110.0000 1.750000 0.875000 100.6250 0.8750000 11.75000 12.125000
## 5         5 108.1818 2.636364 0.454545 268.1818 0.5454545 19.90909 3.181818
##      potass vitamins  shelf  weight    cups  rating
## 1 217.14286 30.35714 2.928571 1.160000 0.6614286 46.01532
## 2  95.00000 10.00000 2.100000 0.883000 0.8640000 60.11492
## 3  80.16129 34.67742 2.161290 1.039032 0.8003226 37.30956
## 4  38.75000 25.00000 2.000000 1.000000 0.8387500 33.65472
## 5  45.90909 31.81818 1.727273 1.000000 1.0345455 42.21039

print(healthy_cluster_index)

## [1] 2

print(healthy_cluster)

##   cluster calories protein fat sodium fiber carbo sugars potass vitamins shelf
## 2         2      86     2.5 0.6      3   2.1 14.6    2.9    95      10   2.1
##   weight cups  rating
## 2  0.883 0.864 60.11492

```

Analysis:

1. As the ward linkage method has the highest value with 0.9046042 it is the best method for the cereals data when compared to all the other methods above. It gives balanced clusters and minimizing the variance within each clusters.
2. Cluster 2 has healthy cereals with low fat, low sodium and high protein when compared to all the other clusters.

Normalizing data: It is important to normalize the data for clustering because the algorithm relies on distance measures between data points. Normalizing ensures that each variable contributes equally when forming clusters. Without normalization, variables with larger scales can have a greater impact on cluster formation and lead to misleading results.